Paydirt

SECURITY ARCHITECTURE

Architecture and Security Review for Compliance Reviewers

How Paydirt and log_scrubber.py handle sensitive data, what trust boundaries exist, and what evidence supports the security claims made about these tools.

DOCUMENT
Security Architecture v1.2.0

DATE
April 26, 2026

PUBLISHER
Machine Data Insights Inc.
There's Gold In That Data!®

Executive Summary

Paydirt is a log scrubbing tool distributed as a single, self-contained HTML file (`Paydirt.html` , approximately 120 KB) and as a paired Python command-line utility (`log_scrubber.py`). Both tools redact sensitive content - including personally identifiable information, protected health information, controlled unclassified information markings, credentials, and tokens - from log data so that sanitized samples can be safely shared with downstream consumers (analysts, large language models, ticketing systems, support teams).

This document is intended for security and compliance professionals evaluating Paydirt for use in their environment. It addresses the questions a reviewer is most likely to ask: what the tool does, where data goes, what attack surface is being accepted, and what evidence supports the claims made about the tool.

> **Key security properties:**
>
> - **Offline by design.** The browser tool makes zero network requests during operation. The Python CLI makes zero network requests at any time.
> - **No external dependencies.** The browser tool uses no CDN-loaded scripts, fonts, or stylesheets. The Python CLI requires only the Python 3.9+ standard library.
> - **Strict Content Security Policy.** The browser tool ships with a CSP that disallows external resources, inline event handlers, and `eval()` .
> - **Source-available under Apache License 2.0.** The entire source is published at `github.com/machinedatainsights/paydirt` for review. The browser tool is built from that source by a simple Python script that performs string concatenation only.
> - **No telemetry, analytics, or external reporting.** The tools do not phone home, do not collect usage statistics, and have no opt-in or opt-out for data collection because no data collection occurs.

Audience and Scope

This document covers the technical architecture, data-handling behavior, and security posture of Paydirt and log_scrubber.py as distributed in release v1.2.0. It does not cover the broader compliance program of any organization deploying the tools - that remains the responsibility of the deploying organization.

Compliance reviewers, application security teams, and procurement professionals are the primary intended audience. Software engineers evaluating the tools for integration into existing systems will also find the architecture sections useful.

Data Flow

The single most important question a reviewer asks is: when I use this tool to scrub a log file, where does my data go? The answer is short: nowhere. Data enters the tool via local file selection, gets processed in memory, and exits the tool via a local download or write. No network transmission occurs at any step.

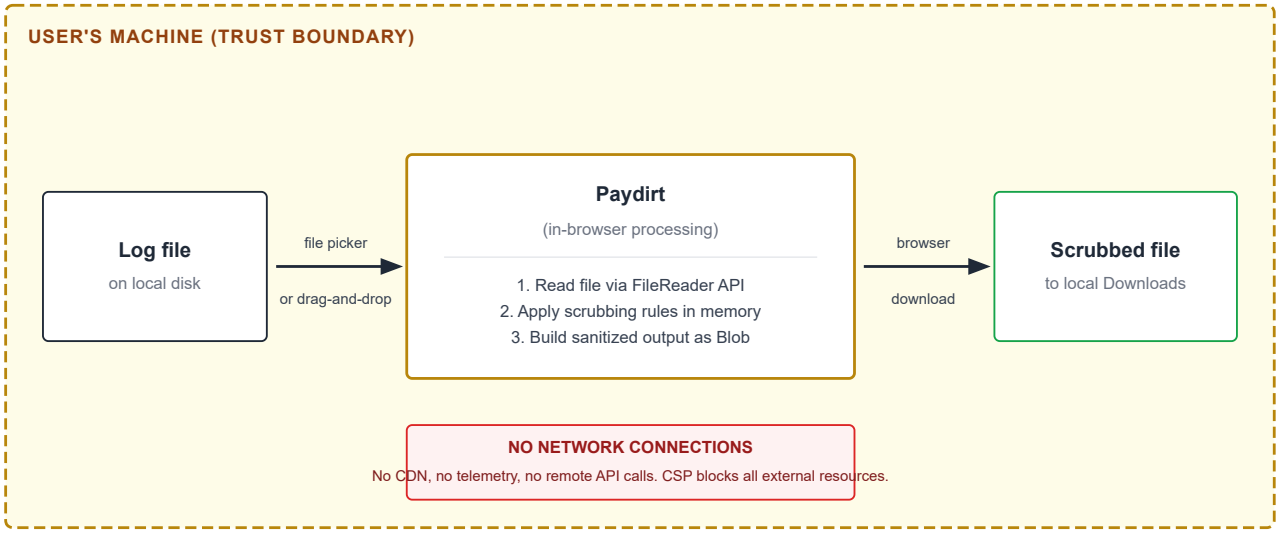

Figure 1. Paydirt data flow. All processing occurs within the browser on the user's machine. No data crosses the trust boundary (the dashed gold border).

Step-by-Step Walkthrough

The user opens `Paydirt.html` by double-clicking the file. The browser loads the page from the local filesystem; no HTTP request to any server occurs. The browser parses the HTML, applies the inline CSS, and executes the inline JavaScript. The page is now interactive.

The user drops a log file onto the page or clicks the file picker. The browser's File API exposes the file contents to the page's JavaScript via the `FileReader` interface. The file is read entirely into memory as text. No network call is made; the File API operates locally.

The page's JavaScript scrubbing engine runs the loaded text through a series of regular expressions (built-in PII, credential, and CUI patterns) and any user-provided configuration rules. The scrubbed text is built as a string in memory, then wrapped in a `Blob` and presented to the user via a generated download link. When the user clicks the download link, the browser saves the Blob to the local Downloads folder using the standard download mechanism.

At no point during this sequence is any HTTP request made. The browser's network stack is not invoked.

The Python CLI

The CLI (`log_scrubber.py`) operates equivalently: it reads files from the local filesystem, processes them in memory using the same scrubbing algorithm, and writes the sanitized output to a local file. The CLI imports only modules from the Python standard library; it does not initiate any network connections.

Architecture

Paydirt is structured as a single-file distribution. The browser tool consists of one HTML file containing all CSS, JavaScript, and a default configuration string. The Python CLI consists of one Python file. There are no installers, no package managers, no runtime dependencies beyond a modern web browser (for Paydirt) or Python 3.9 or later (for the CLI).

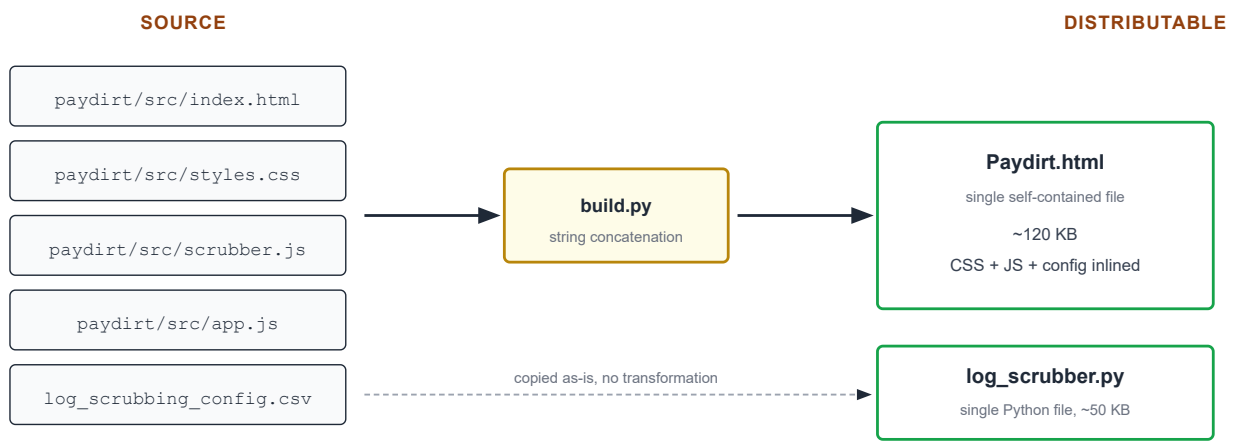

Figure 2. Build process. The browser tool's source files are concatenated into a single self-contained HTML file. The Python CLI is shipped as-is from source. No bundlers, transpilers, or minifiers are involved.

What's Inside Paydirt.html

The single file `Paydirt.html` contains:

- The HTML structure for the user interface (drop zone, configuration section, results card with three tabs).
- All CSS styling, inlined inside a `<style>` tag. No external stylesheets, no web fonts.

- The scrubbing engine (`scrubber.js` contents), inlined inside a `<script>` tag. This is a JavaScript port of `log_scrubber.py` that produces byte-identical output for identical input (verified by the parity test suite).
- The user interface controller (`app.js` contents), inlined inside another `<script>` tag. Handles file selection, drag-and-drop, configuration loading, and result rendering.
- The default scrubbing configuration (`log_scrubbing_config.csv`), inlined as a JavaScript string literal so the tool has working defaults out of the box.

Because the file is fully self-contained, it can be loaded directly from the local filesystem (`file://` URL) without any web server. This is the recommended deployment mode and the one all security claims in this document apply to.

Build Process and Reproducibility

Anyone with Python 3 installed can rebuild `Paydirt.html` from source by running `python3 paydirt/build.py` from the repository root. The build script is approximately 80 lines of Python that reads the four source files, the default config CSV, and produces the output file via string substitution into the HTML template. There is no minification, transpilation, or other transformation that would obscure the relationship between source and output.

A reviewer can run the build script themselves and compare the result to the published `Paydirt.html` from the GitHub release. Differences should be limited to whitespace and version strings. If the rebuilt file differs substantively from the published one, that is grounds for raising a concern with Machine Data Insights.

Security Properties

The following properties are deliberate design choices, not accidental side effects of the implementation. Each is verifiable by inspecting the source.

No External Resources

`Paydirt.html` contains no references to external URLs that would trigger network loads. There are no CDN-loaded scripts, no Google Fonts, no analytics pixels, no remote stylesheets, and no third-party iframes. The file's `<head>` contains only inlined `<style>` and `<script>` blocks plus a single inline SVG favicon as a data URL.

The only outbound URLs that appear anywhere in the file are documentation references in HTML attributes (`href`) on a few hyperlinks, such as the link to the Apache License text in the footer. These links are not auto-loaded; they activate only if the user clicks them.

Strict Content Security Policy

The page declares a Content Security Policy via meta tag that prohibits loading external scripts, styles, frames, fonts, images, and other resources. Inline event handlers (`onclick` , etc.) are also prohibited. The CSP allows only the resources that are inlined into the HTML file itself.

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'unsafe-inline';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data:;
  connect-src 'none';
  frame-src 'none';
  object-src 'none';
  base-uri 'none';
  form-action 'none';
```

The `'unsafe-inline'` directive on script-src and style-src is required because Paydirt's CSS and JavaScript are inlined in the HTML file. This is intentional: the alternative would be loading those resources from external files, which violates the no-external-resources property. Inline content from a same-origin file is the safest available arrangement when external loading is forbidden.

The `connect-src 'none'` directive is the most important line for a security reviewer: it explicitly prohibits the page from making any network connections via `fetch()` , `XMLHttpRequest` , WebSocket, or similar APIs. Even if the JavaScript code were modified to attempt a network call, the browser would block it.

No eval, no Function constructor

Neither `scrubber.js` nor `app.js` uses `eval()` , the `Function` constructor, or any other dynamic code-execution mechanism. The CSP enforces this by omitting the `'unsafe-eval'` directive: any attempt to call `eval()` would be blocked by the browser.

No Persistent Storage

Paydirt does not use cookies, `localStorage` , `sessionStorage` , IndexedDB, or any other browser-side persistent storage mechanism. Each session starts fresh; closing the tab discards all loaded data.

There is one exception: if the user explicitly checks the "Remember this config on this browser" checkbox in the configuration panel, Paydirt will store the loaded configuration text (only the config, not any log data) in `localStorage` so it persists across sessions. This is opt-in, off by default, and clearly labeled. No log content is ever stored persistently regardless of this setting.

No Telemetry

There is no analytics code, no error reporting, no usage tracking, no crash reporting, no A/B testing infrastructure, and no opt-in or opt-out mechanism for any of the above because none exists. Paydirt cannot phone home because there is no home to phone to.

Threat Model

Honest scope-setting matters more than feature claims. This section identifies what Paydirt protects against and what it does not.

What Paydirt Protects Against

PROTECTS AGAINST: Inadvertent disclosure during sample sharing

The most common compliance risk in normal Splunk engineering work is a well-meaning engineer pasting a log sample into a ticket, email, or LLM prompt without realizing the sample contains real PII or credentials. Paydirt's primary purpose is to interpose a fast, reliable scrubbing step between extraction and sharing.

PROTECTS AGAINST: Network-borne data exfiltration via the tool itself

Because Paydirt makes no network connections, it cannot be the channel by which scrubbed or unscrubbed data leaves the user's machine. Even a hypothetical compromise of the source code could not reintroduce network connectivity without also defeating the CSP, which would be detectable by inspection.

PROTECTS AGAINST: Supply-chain compromise of dependencies

Paydirt has no third-party dependencies. There are no npm packages, no pip packages beyond the Python standard library, no CDN-loaded scripts. A supply-chain attack on the JavaScript or Python ecosystems cannot reach Paydirt's runtime because Paydirt does not consume from those ecosystems.

PROTECTS AGAINST: Cross-environment data leakage

Files processed by Paydirt remain on the user's machine. They are not uploaded to a SaaS, cached on a vendor's server, or processed in a multi-tenant environment. Single-machine processing is the entirety of the data plane.

What Paydirt Does Not Protect Against

DOES NOT PROTECT AGAINST: A compromised browser or operating system

If the user's browser is compromised by malware, or the underlying operating system is compromised, scrubbing performed in that browser is no more trustworthy than any other operation performed on a compromised system. Paydirt assumes a trustworthy execution environment.

DOES NOT PROTECT AGAINST: Sensitive data without recognizable patterns

Paydirt's built-in patterns target well-defined formats: IP addresses, email addresses, credit card numbers, etc. Sensitive content that doesn't match these patterns - for example, a free-text incident description containing a customer's real name - will pass through unredacted unless the user has configured a custom rule for it.

DOES NOT PROTECT AGAINST: Misuse by an authorized user

A user with legitimate access to sensitive logs who chooses not to use Paydirt before sharing data is not constrained by Paydirt. The tool is a control that the user must choose to apply. Defense in depth requires complementary controls (data loss prevention, transmission monitoring, access logging) outside the scope of this tool.

DOES NOT PROTECT AGAINST: CUI content without CUI markings

Paydirt's CUI redaction is triggered by the presence of marking text (CUI banners, FOUO tags, ITAR designations, and so on). Logs containing CUI-regulated content that has not been marked as such will not trigger whole-line redaction, though pattern-based redactions (PII, credentials) still apply to recognizable elements within those logs.

Compliance Notes

Paydirt is a scrubbing utility, not a compliance product. It supports compliance workflows in specific ways that are worth being explicit about so reviewers can correctly position the tool in their broader compliance program.

For CMMC and NIST SP 800-171 environments, Paydirt's CUI marking detection helps prevent inadvertent disclosure of CUI content during routine engineering activities (analyzing logs, sharing samples for troubleshooting, building search queries). The tool recognizes the four major marking styles: banner markings (CUI, CUI//SP-*), portion markings (parenthesized classifications), legacy markings (FOUO, SBU, LES, others), and adjacent classifications (ITAR, EAR99, ECCN). When these markings are detected, the entire containing line is redacted with a placeholder that preserves marking category and original byte count for downstream visibility.

For HIPAA-covered entities, Paydirt redacts the Safe Harbor identifier categories that have machine-detectable shape: Social Security Numbers (with valid-range filtering to avoid false positives on invalid ranges), Luhn-validated credit card numbers, Luhn-validated National Provider Identifiers (per 45 CFR 162.406), formatted phone numbers, email addresses, and IP addresses. PHI identifiers without

recognizable structure (full names in free text, medical record numbers in arbitrary formats) require organization-specific custom rules.

For GDPR-relevant data, the same pattern detection handles the personal data categories with recognizable shape: IP addresses, email addresses, device identifiers (MAC addresses), and user identifiers in well-defined formats. Free-text personal data requires custom rules.

In all three cases, Paydirt is one layer in a defense-in-depth program. The tool's value is in catching the routine, pattern-matchable disclosures that account for most accidental data leaks. It is not a substitute for organizational controls, training, or transmission monitoring.

Code Provenance and Verification

The full source for both Paydirt and log_scrubber.py is published at:

```
https://github.com/machinedatainsights/paydirt
```

Released artifacts are available at the GitHub Releases page for the repository. Each release includes the built `Paydirt.html`, the `log_scrubber.py` source, the default configuration file, and the demo log file used to exercise every scrubbing category.

Verifying the Distributed Build

To verify that the published `Paydirt.html` was built from the published source:

1. Clone the repository at the tag corresponding to the release version.
2. Run `python3 paydirt/build.py` from the repository root.
3. Compare the generated `Paydirt.html` to the one downloaded from the GitHub release. They should be byte-identical except for any version-string substitutions performed at build time.

The build script is short (approximately 80 lines) and performs only file reads and string concatenation. A reviewer can read it in under five minutes and verify there are no obfuscation steps, no remote fetches, no minification.

Verifying Behavioral Equivalence Between Tools

The repository includes a parity test suite at `shared-tests/run_parity.py` that runs identical input fixtures through both the Python CLI and the JavaScript implementation (via Node.js) and verifies the outputs match. This guards against drift between the two implementations: a user who scrubs a file with Paydirt should get the same result as scrubbing it with log_scrubber.py.

Test Coverage

The repository includes three test suites that together cover the behavioral surface of the scrubbing engine:

Test suite	Location	What it verifies
Python CLI tests	`cli/test_scrubber.py`	50 unit tests covering config parsing, all built-in patterns, CUI detection, JSON path traversal, library API.
JavaScript scrubber tests	`paydirt/tests/test-node.js`	50 unit tests mirroring the Python suite, run via Node.js to verify the JavaScript implementation produces identical results.
Browser smoke tests	`paydirt/tests/smoke-test.py`	End-to-end browser tests using Playwright. Loads the built Paydirt.html in a headless browser, simulates user interactions, verifies no console errors and zero outbound network requests.
Parity tests	`shared-tests/run_parity.py`	Byte-for-byte comparison of Python CLI output vs JavaScript output on shared input fixtures. Detects implementation drift.

All test suites run in seconds and produce deterministic pass/fail output. A reviewer can verify the build is functioning correctly by running them on their own machine.

License and Trade Marks

Paydirt and log_scrubber.py are licensed under the Apache License, Version 2.0. The full license text is included in the repository as `LICENSE`, and attribution requirements (including a request that forks not use the Paydirt or Machine Data Insights names) are documented in `NOTICE`.

"Paydirt", "Machine Data Insights", "There's Gold In That Data!", and "Turning Data Into Gold" are trade marks of Machine Data Insights Inc. The license grants rights to the code; it does not grant rights to use these names for derived works.

Document version: 1.2.0 (April 26, 2026). This document is distributed alongside Paydirt v1.2.0 and is updated when significant architectural or security-relevant changes are made to the tool.

Reporting concerns: Security concerns about Paydirt or this document should be sent to the contact address listed at `machinedatainsights.com`. Please do not report security issues via public GitHub Issues, which are disabled on this repository.

Machine Data Insights Inc. *There's Gold In That Data!*® | `machinedatainsights.com`